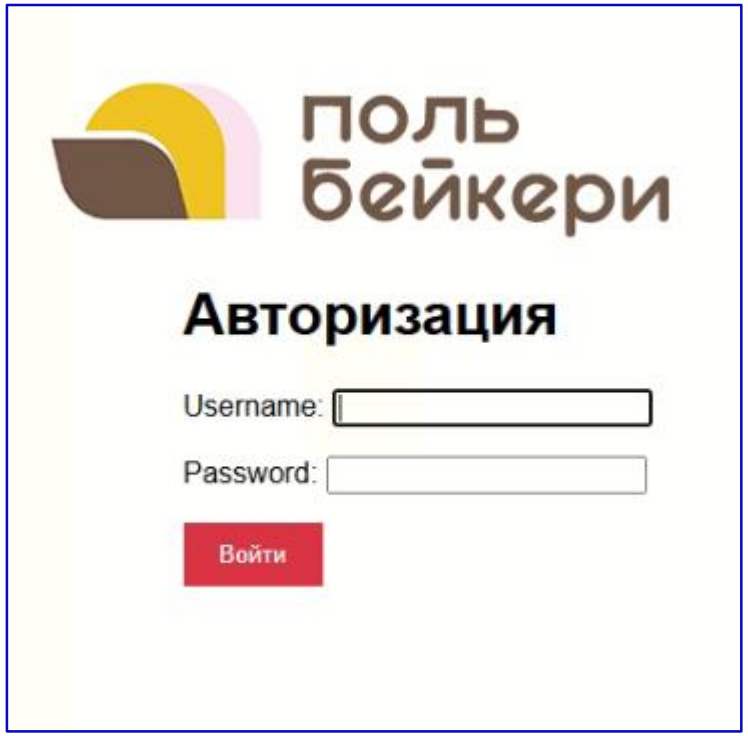
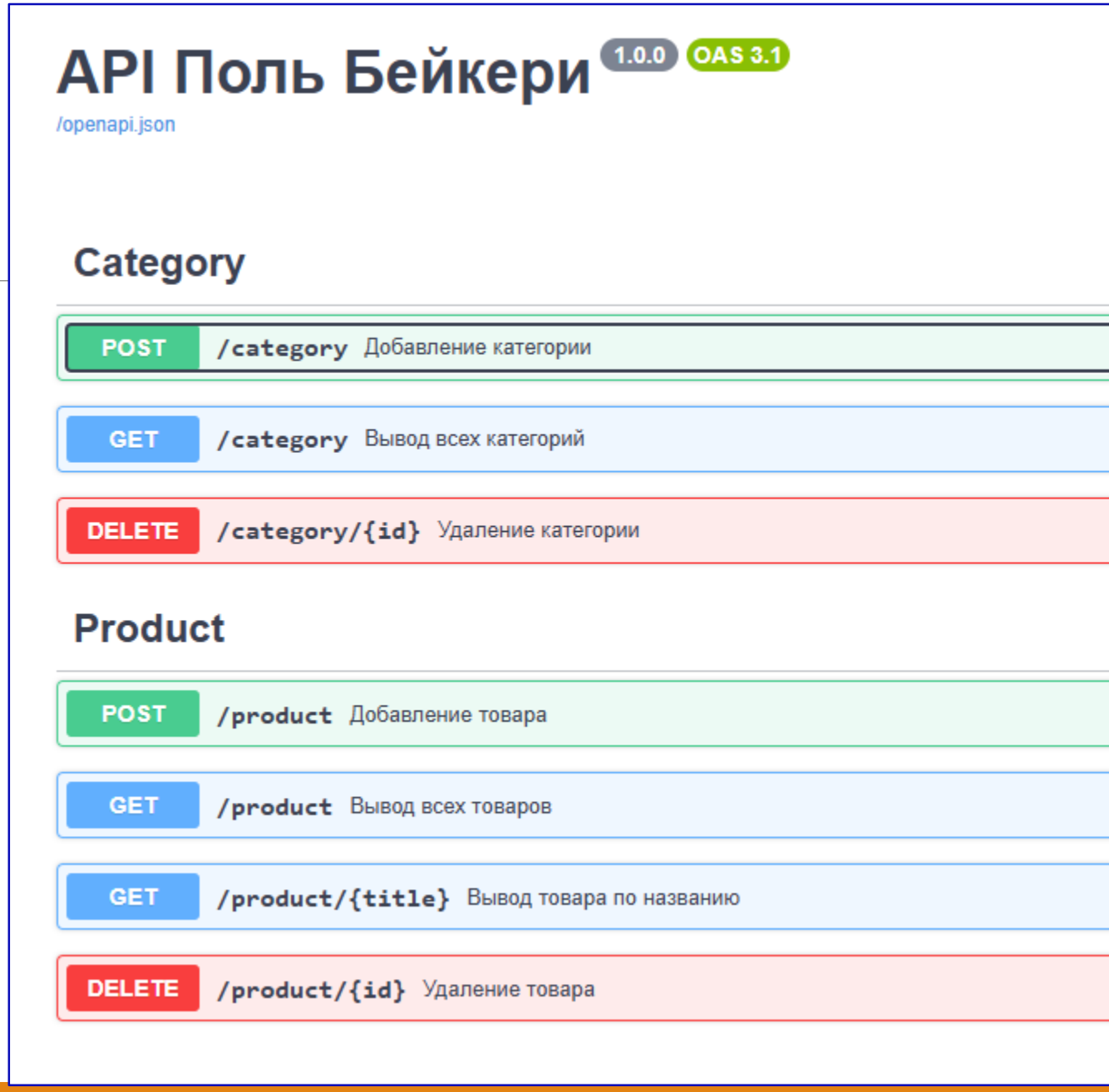


Админ-панель Web-приложения “Поль бейкери”

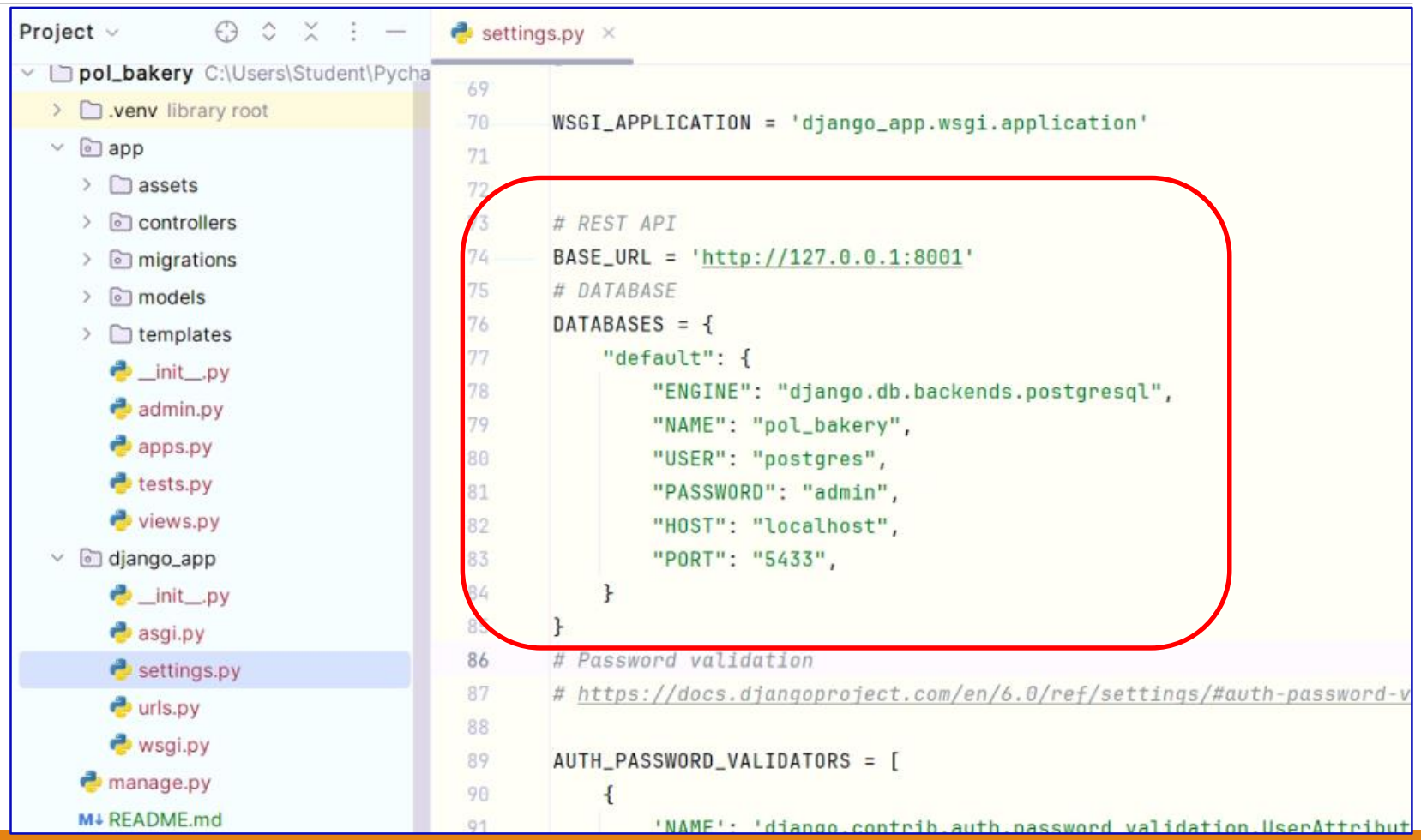
API

Создадим классическую Django admin – панель , использующую внешнее API.



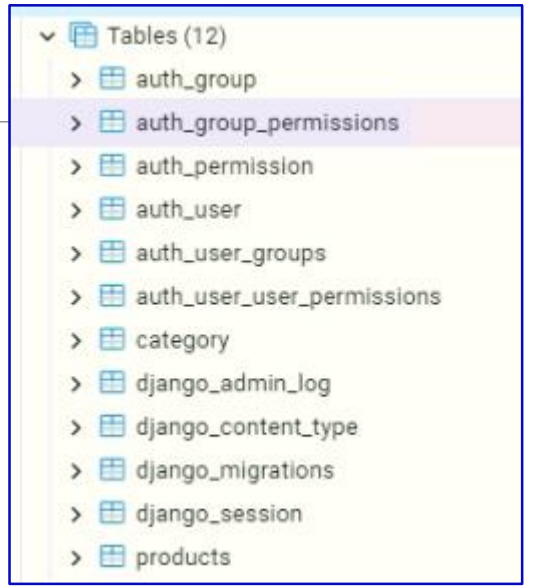
В настройках Django- проекта укажите url-адрес API и базу данных для админ-панели

Подключение базы данных выполняется, в связи с тем, что в API отсутствует методы авторизации пользователей (при наличии методов авторизации в API необходимо использовать только API)



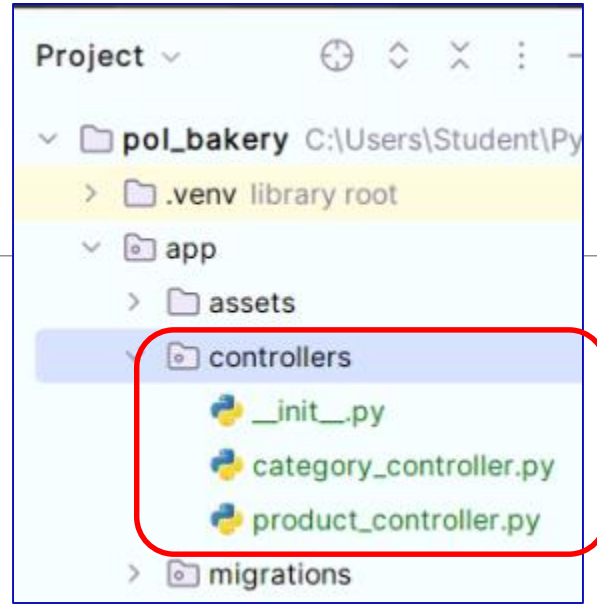
«Включите админку» и создайте суперпользователя

```
venv\Scripts\activate
(venv) $ python manage.py migrate
(venv) $ python manage.py createsuperuser
(venv) $ python manage.py runserver
```




```
(.venv) PS C:\Users\Student\PycharmProjects\pol_bakery> python manage.py createsuperuser
Username (leave blank to use 'student'): admin
Email address: admin@mail.ru
Password:
Password (again):
The password is too similar to the username.
This password is too short. It must contain at least 8 characters.
This password is too common.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

Контроллеры



Методы для админ-панели (работа с категориями)

category_controller.py

```python
import requests
from django_app import settings
base_url=settings.BASE_URL

7 usages  new *
class CategoryController():
    new *
    @staticmethod
    def create_category(data):
        """Создание категории"""
        resp = requests.post( url: f"{base_url}/category", json=data)
        resp.raise_for_status()
        return resp.json()

    1 usage  new *
    @staticmethod
    def delete_category(id_category: int):
        """Удаление категории"""
        resp = requests.delete(f"{base_url}/category/{id_category}")
        resp.raise_for_status()
        return None

    3 usages  new *
    @staticmethod
    def get_categories(page=1, size=50):
        '''    Вывод категорий      '''
        response = requests.get( url: f"{base_url}/category", params={"page": page, "size": size})
        response.raise_for_status()
        categories = response.json()['items']
        return categories
```

Category

POST	/category	Добавление категории
GET	/category	Вывод всех категорий
GET	/category/{route_category}	Вывод категории по url-маршруту
DELETE	/category/{id}	Удаление категории

Методы для админ-панели (работа с товарами)

product_controller.py

```python
import requests
from django_app import settings
base_url=settings.BASE_URL

4 usages  new *
class ProductController():
    new *
    @staticmethod
    def create_product(data):
        """Создание товара"""
        response = requests.post( url: f"{base_url}/product", json=data)
        response.raise_for_status()
        return response.json()

    new *
    @staticmethod
    def delete_product(id_product: int):
        """Удаление товара"""
        response = requests.delete(f"{base_url}/product/{id_product}")
        response.raise_for_status()
        return None

    1 usage  new *
    @staticmethod
    def get_products(page=1, size=50):
        '''   Вывод товаров    '''
        response = requests.get( url: f'{base_url}/product', params={"page": page, "size": size})
        response.raise_for_status()
        products = response.json()['items']
        return products
```

Product

POST	/product	Добавление товара
GET	/product	Вывод всех товаров
GET	/product/{route_category}	Получить товары по по url-маршруту категории
DELETE	/product/{id}	Удаление товара

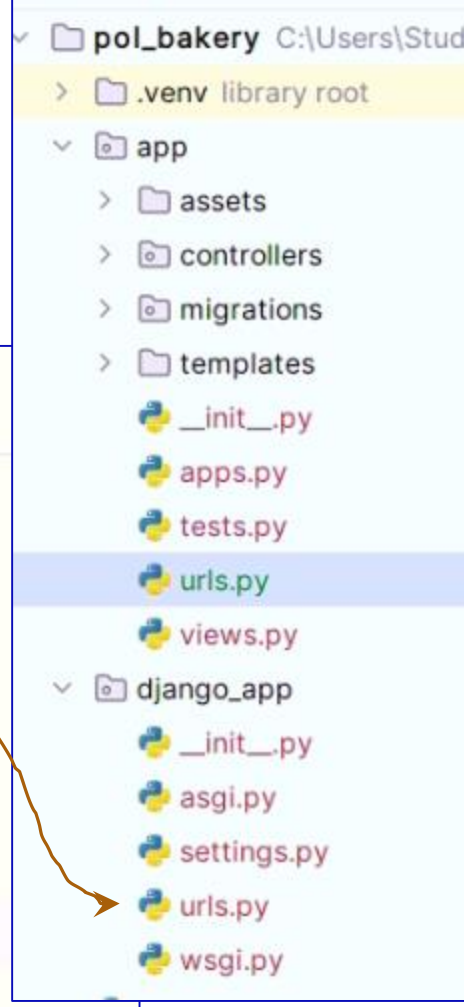
django_app/urls.py
(базовые маршруты)

django_app\urls.py

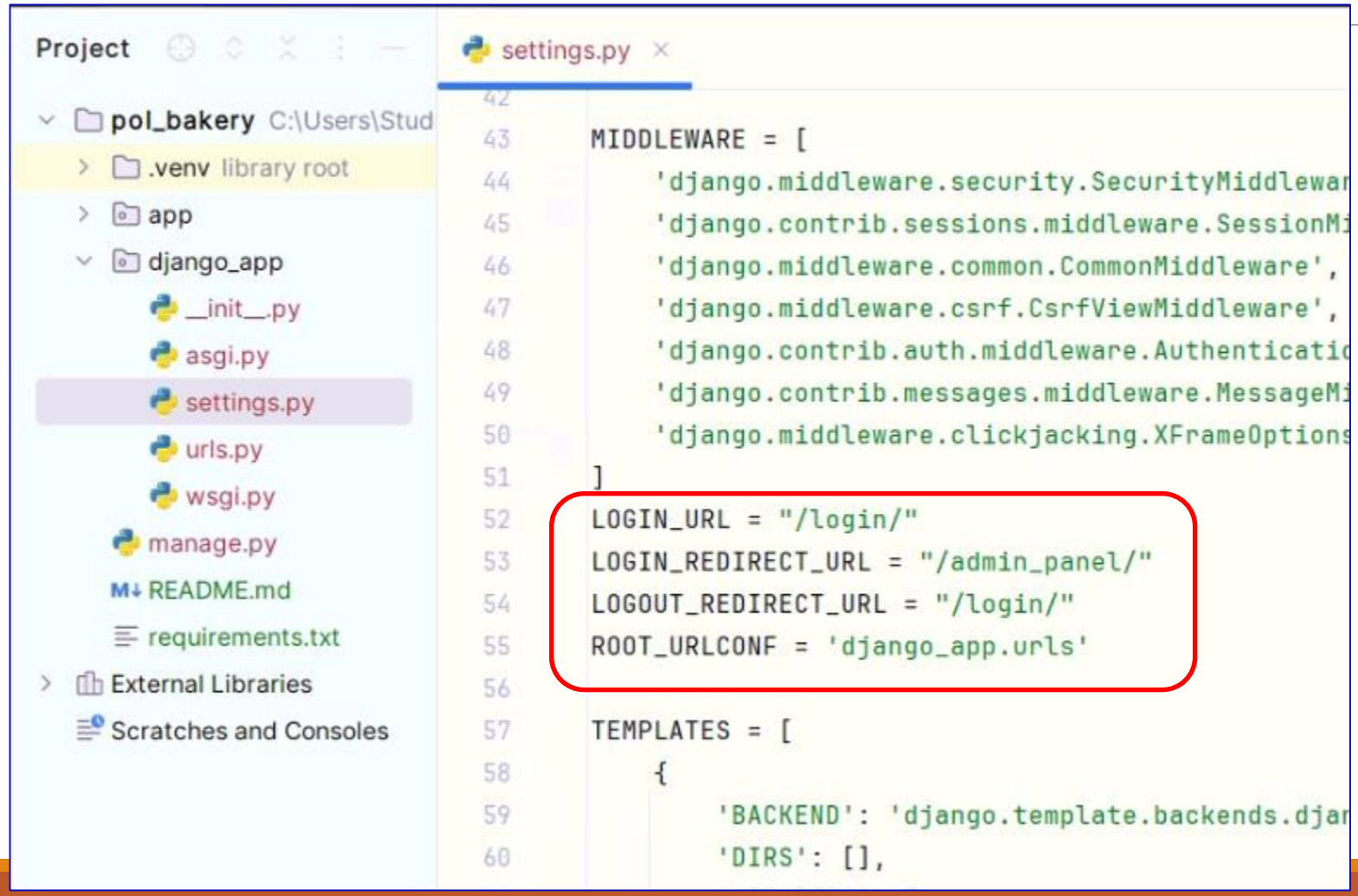
```python
from django.urls import path, include
from django.contrib.auth import views as auth_views
from app import views

urlpatterns = [
    path("admin_panel/", include("app.urls")),  # кастомная админ-панель
    path("login/", auth_views.LoginView.as_view(), name="login"),
    path("logout/", views.logout_view, name="logout"),
    # маршрут срабатывает при открытии сайта
    path('', views.index, name='index'),
    # маршрут срабатывает при выборе категории
    path('products/<str:route_category>/', views.category_data, name='category_data'),
]
```

Маршруты админ-панели находятся в файле app.urls.py (создайте файл urls.py в папке app)



Настройки проекта



LOGIN_URL = "/login/"
– задает страницу авторизации

LOGIN_REDIRECT_URL = "/admin_panel/"
- автоматический переход на главную страницу админ-панели

LOGOUT_REDIRECT_URL = "/login/"
- метод "/logout/" выполняет автоматический переход на маршрут "/login/"

ROOT_URLCONF = 'django_app.urls‘
- Базовые маршруты

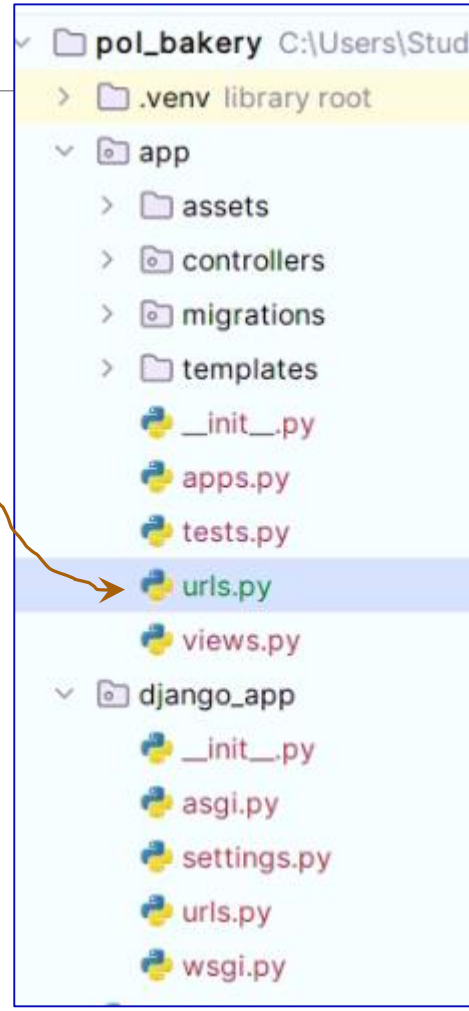
app/urls.py
(добавим маршруты админ-панели)

app\urls.py

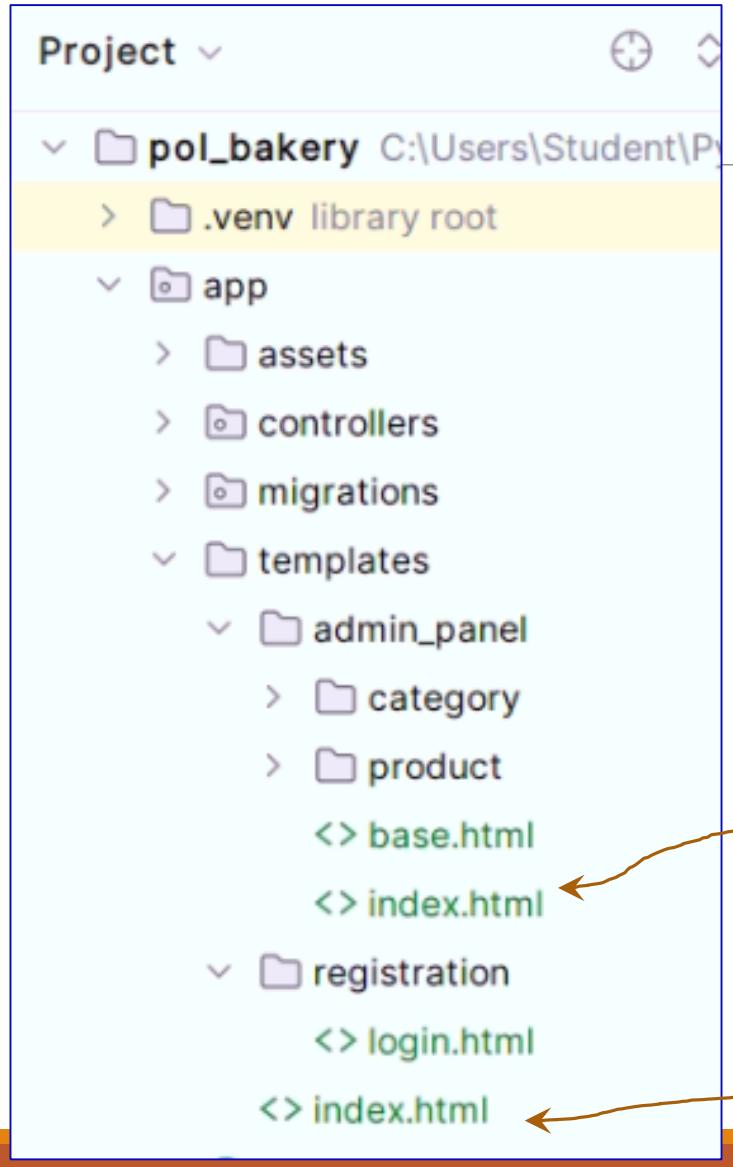
```python
from django.urls import path

from app import views
app_name = "admin_panel"
urlpatterns = [
    # маршруты админ-панели
    path("", views.index_admin_panel, name="index_admin_panel"),
    path("categories/", views.category_list, name="category_list"),
    path("categories/create/", views.category_create, name="category_create"),
    path("categories/<int:pk>/delete/", views.category_delete, name="category_delete"),

    path("products/", views.product_list, name="product_list"),
    path("products/create/", views.product_create, name="product_create"),
    path("products/<int:pk>/delete/", views.product_delete, name="product_delete"),
]
```



Архитектура файлов



В папках templates/category и templates/product находятся страницы для администрирования таблиц category и product

Файл templates/admin_panel/index.html выполняет переход на главную страницу админ-панели сайта

Файл templates/index.html выполняет переход на посадочную страницу сайта

views.py

@login_required — блокирует неавторизованных пользователей (функция работает только для авторизованных пользователей)

views.py

```
23  from django.contrib.auth import logout
24
    1 usage
25  @login_required
26  def index_admin_panel(request):
27      return render(request, template_name: "admin_panel/index.html")
28
    1 usage
29  @login_required
30  def logout_view(request):
31      logout(request)
32      return redirect("login")
33
```

app\urls.py

```
1  from django.urls import path
2
3  from app import views
4  app_name = "admin_panel"
5  urlpatterns = [
6      # маршруты админ-панели
7      path("", views.index_admin_panel, name="index_admin_panel"),
```

views.py

@login_required — блокирует неавторизованных пользователей
(функция работает только для авторизованных пользователей)

views.py

```
23  from django.contrib.auth import logout
24
    1 usage
25  @login_required
26  def index_admin_panel(request):
27      return render(request, template_name: "admin_panel/index.html")
28
    1 usage
29  @login_required
30  def logout_view(request):
31      logout(request)
32      return redirect("login")
33
```

django_app\urls.py

```
from django.urls import path, include
from django.contrib.auth import views as auth_views
from app import views

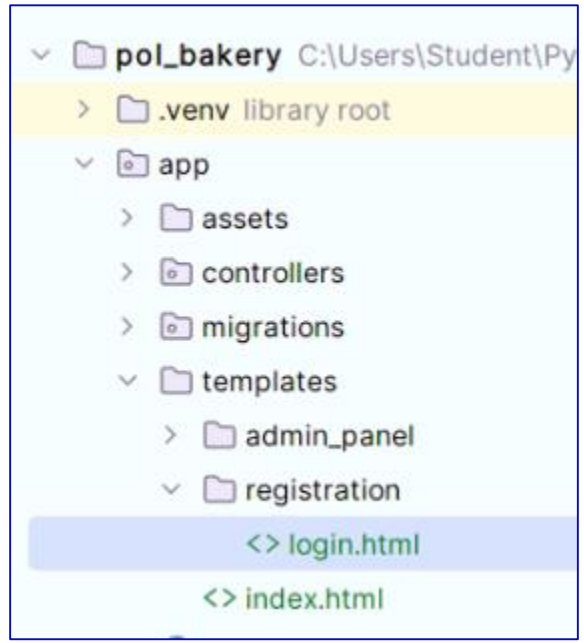
urlpatterns = [
    path("admin_panel/", include("app.urls")),  # кастомная админ-панель
    path("login/", auth_views.LoginView.as_view(), name="login"),
    path("logout/", views.logout_view, name="logout"),
```

Страница авторизации
templates/registration/login.html

django_app\urls.py

```python
from django.urls import path, include
from django.contrib.auth import views as auth_views
from app import views

urlpatterns = [
    path("admin_panel/", include("app.urls")),  # кастомная админ-панель
    path("login/", auth_views.LoginView.as_view(), name="login"),
    path("logout/", views.logout_view, name="logout"),
```

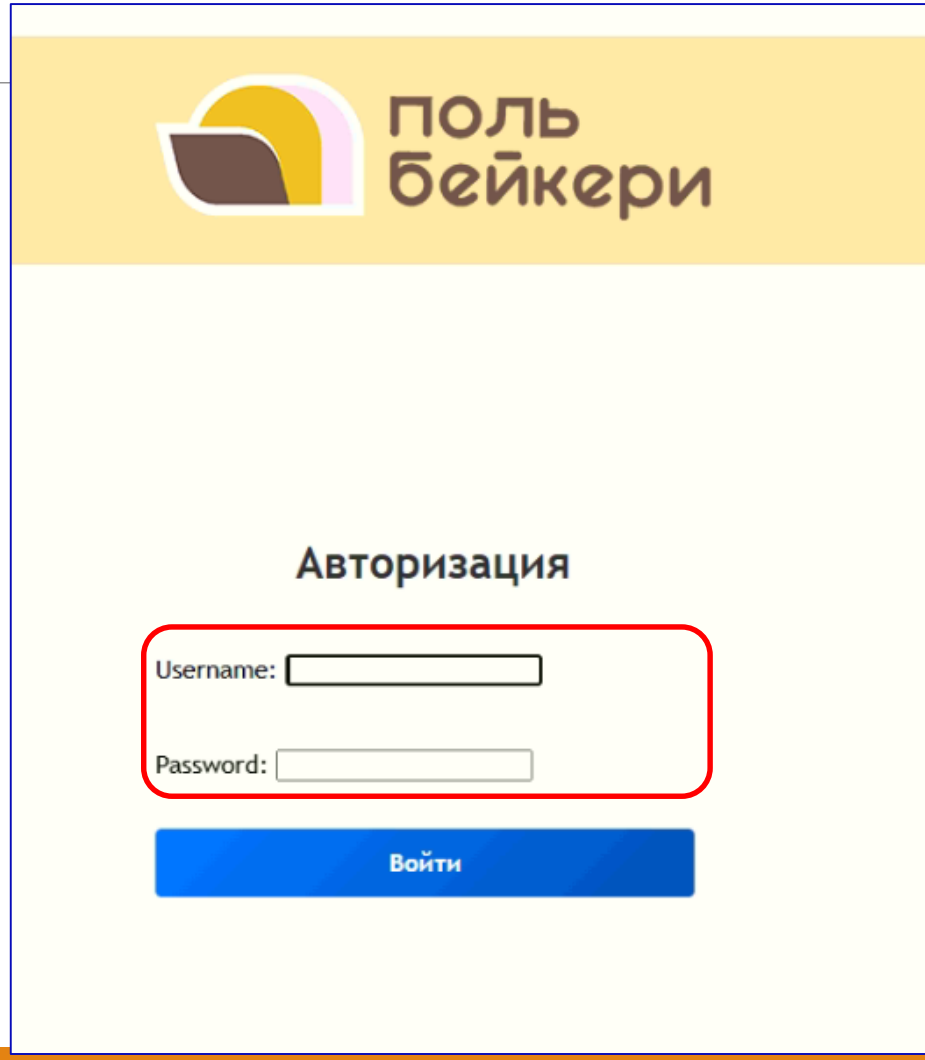


auth_views.LoginView — "умная" встроенная view (нет необходимости создавать метод в файле view.py для функции “login/”)
Реализуется автоматический поиск шаблона в папке templates/registration/login.html

Страница авторизации templates/registration/login.html (вариант 1 – Django-шаблон)

login.html

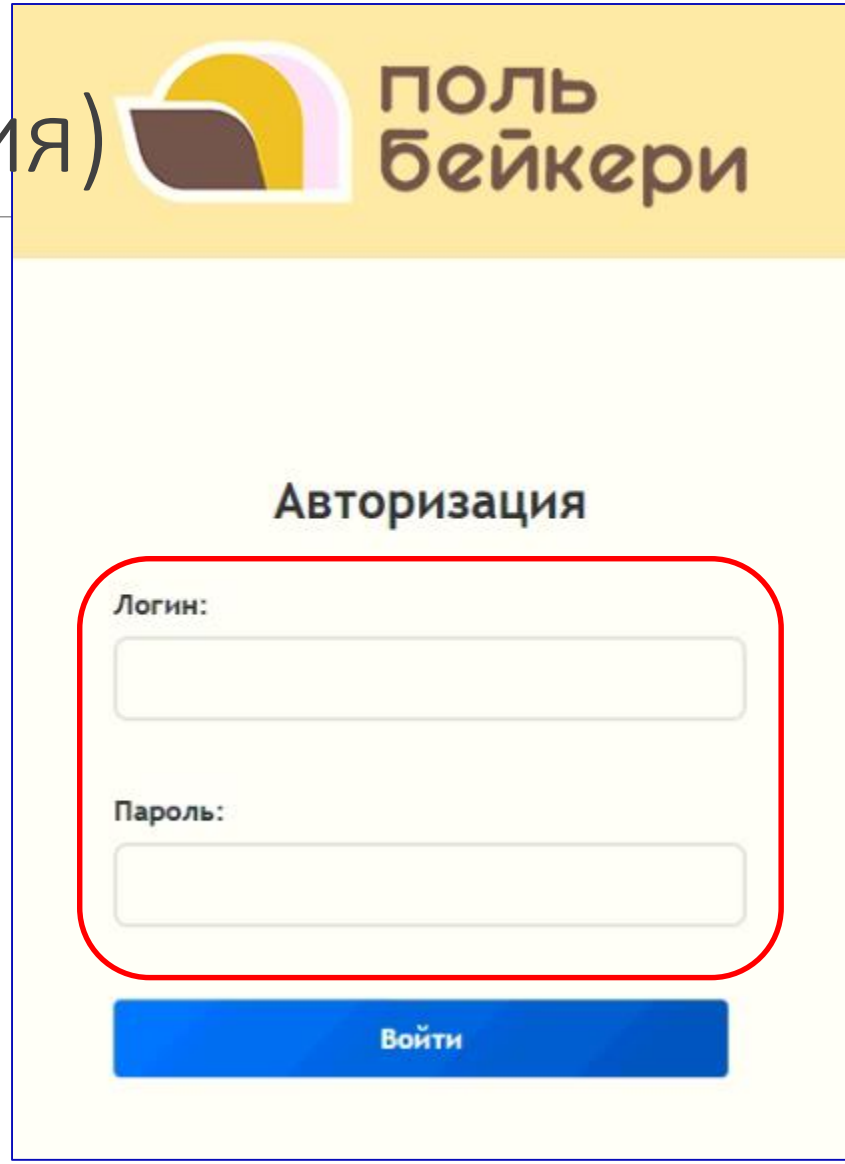
```
1  {%  extends '../admin_panel/base.html' %}
2  {% block title %}Авторизация{% endblock %}
3   {% block content %}
4  <div class="login-container">
5      <h1>Авторизация</h1>
6      <form method="post" class="login-form">
7          {% csrf_token %}
8          {{form.as_p}}
9          <button type="submit">Войти</button>
10     </form>
11 </div>
12
13 {% endblock %}
14
```



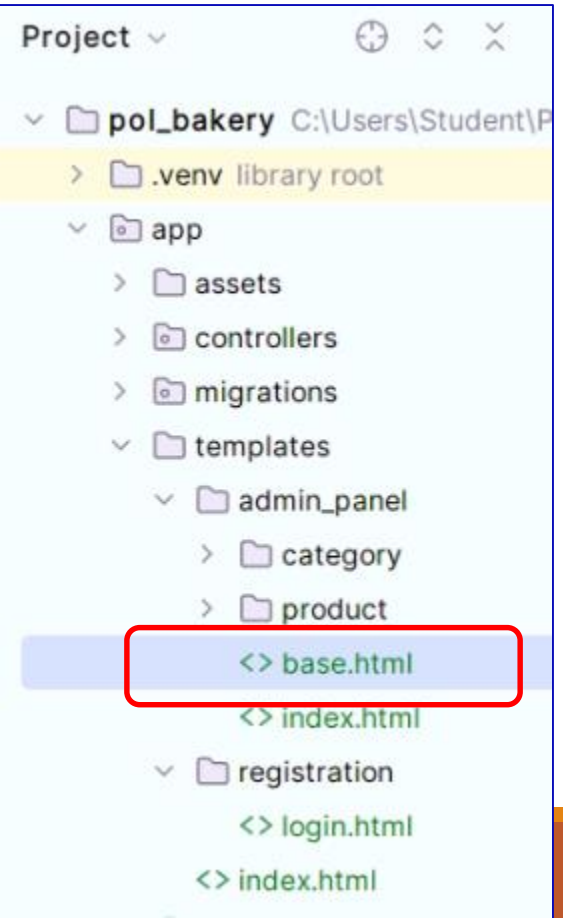
Страница авторизации
templates/registration/login.html
(вариант 2 – кастомная авторизация)

login.html

```
{%  extends '../admin_panel/base.html' %}
{% block title %}Авторизация{% endblock %}
 {% block content %}
<div class="login-container">
    <h1>Авторизация</h1>
    <form method="post" class="login-form">
        {% csrf_token %}
        <div class="form-group">
            <label for="id_username">Логин:</label>
            <input type="text" name="username" id="id_username" required>
        </div>
        <div class="form-group">
            <label for="id_password">Пароль:</label>
            <input type="password" name="password" id="id_password" required>
        </div>
        <button type="submit">Войти</button>
    </form>
</div>
{% endblock %}
```

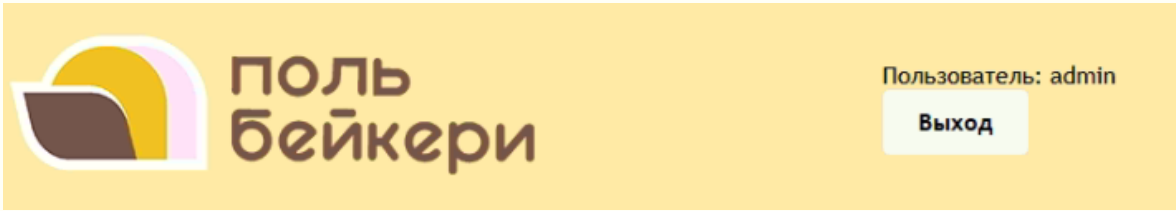


Базовая страница base.html



base.html

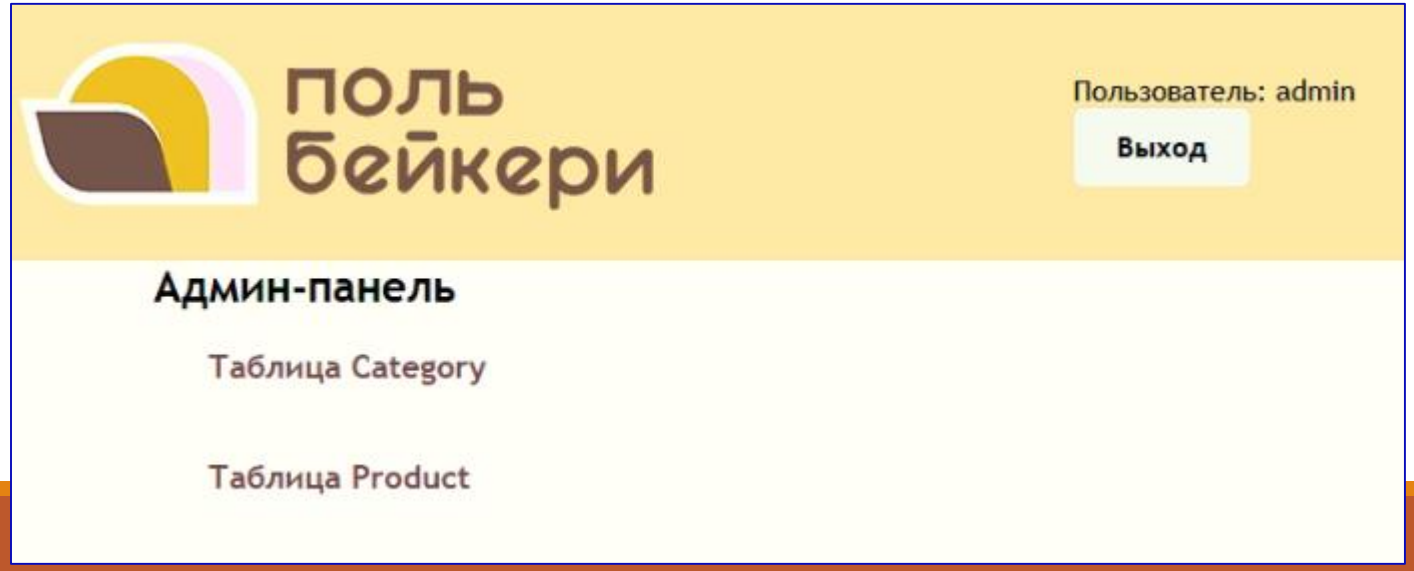
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
<title>{% block title %}Админ-панель{% endblock %}</title>
     {% load static %}
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
<header>
     <div class="logo"><img src="{% static 'images/logo.png' %}" alt="Логотип"></div>
    {% if  user.username %}
        <div class="user-block">
            <span>Пользователь: {{ user.username }}</span>
            <form method="post" action="{% url 'logout' %}" >
                {% csrf_token %}
                <button type="submit"> Выход  </button>
            </form>
        </div>
     {% endif %}
</header>
<main>
    <div class="container">
     {% block content %} {% endblock %}
    </div>
</main>
</body>
</html>
```

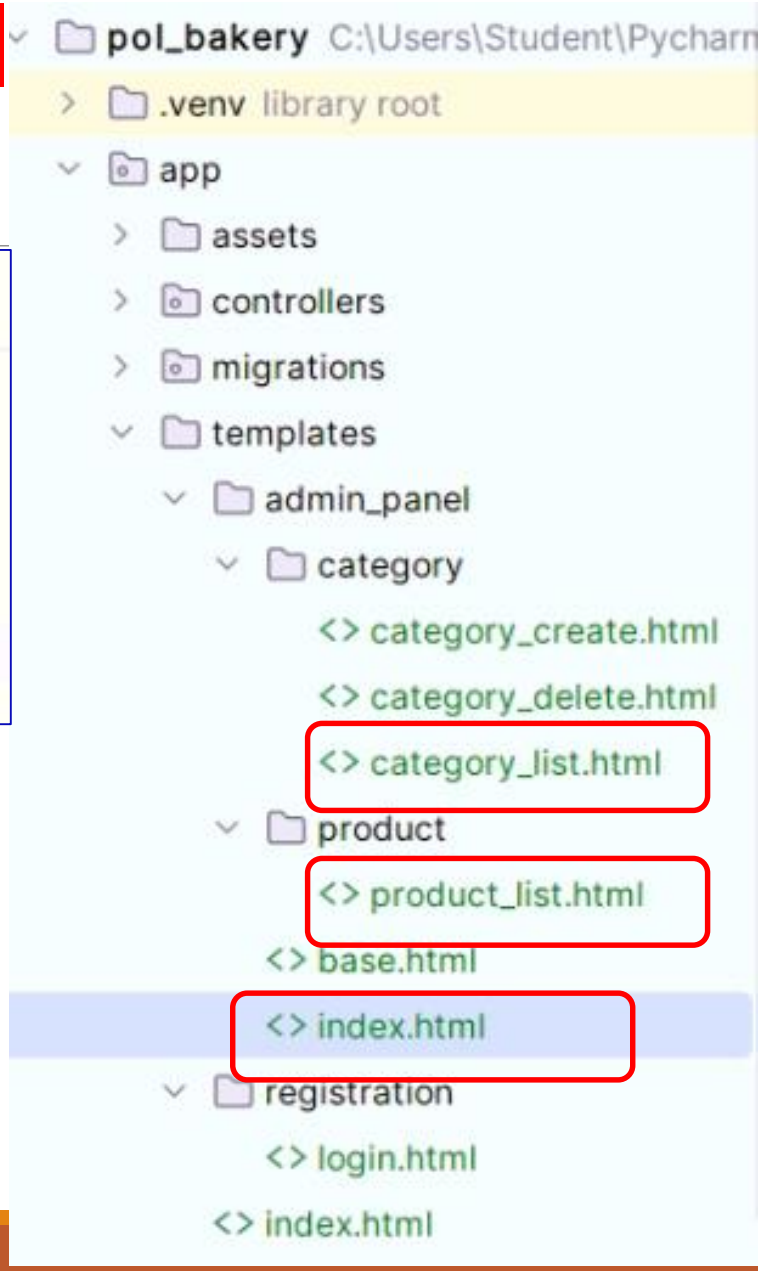


Главная страница admin-панели templates/admin_panel/index.html (при успешной авторизации)

index.html

```
{%  extends 'admin_panel/base.html' %}
{% block content %}
<h2>Админ-панель</h2>
<p><a href="{% url 'admin_panel:category_list' %}">Таблица Category</a></p>
<p><a href="{% url 'admin_panel:product_list' %}">Таблица Product</a></p>
{% endblock %}
```





Страница admin-панели для работы с категориями (вывод категорий из API)

views.py

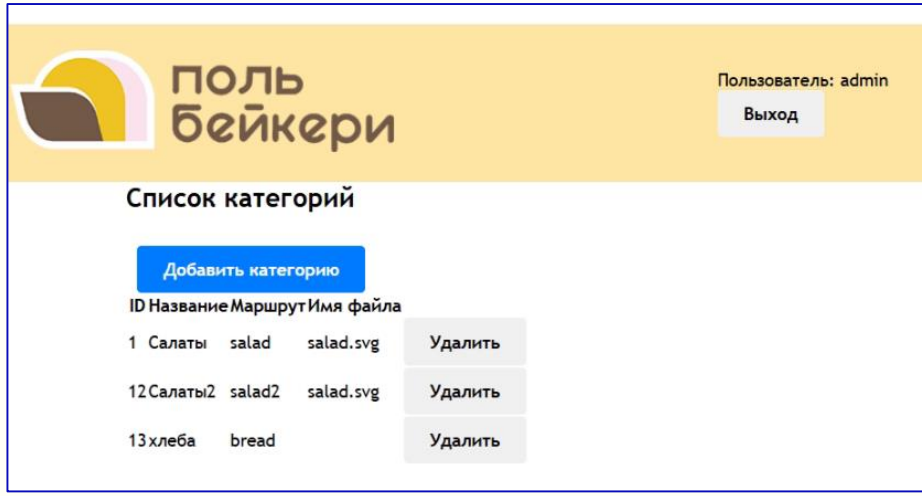
```
1 usage                                                          ⚠ 7  ✓ 13
@login_required
def category_list(request):
    categories = CategoryController.get_categories()
    return render(request, template_name: "admin_panel/category/category_list.html", context: {'items': categories})

```

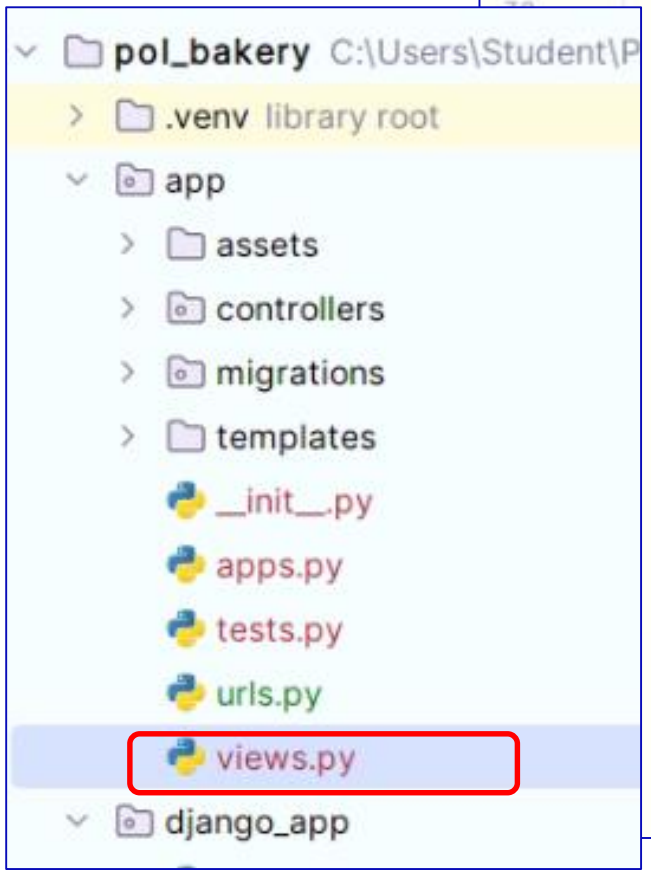
Страница вывода категорий

<> category_list.html

```
{%  extends 'admin_panel/base.html' %}
{% block title %}Список категорий{% endblock %}
{% block content %}
<h2>Список категорий</h2>
<div class="buttons">
    <a class="btn btn-primary" href="{% url 'admin_panel:category_create' %}">Добавить категорию</a>
</div>
<table>
    <tr>
        <th>ID</th>
        <th>Название</th>
        <th>Маршрут</th>
        <th></th>
    </tr>
    {% for c in items %}
    <tr>
        <td>{{ c.id_category }}</td>
        <td>{{ c.name_category }}</td>
        <td>{{ c.route_category }}</td>
        <td>
            <form method="post" action="{% url 'admin_panel:category_delete' c.id_category %}">
                {% csrf_token %}
                <button type="submit">Удалить</button>
            </form>
        </td>
    </tr>
    {% endfor %}
</table>
{% endblock %}
```



Удаление категории



views.py

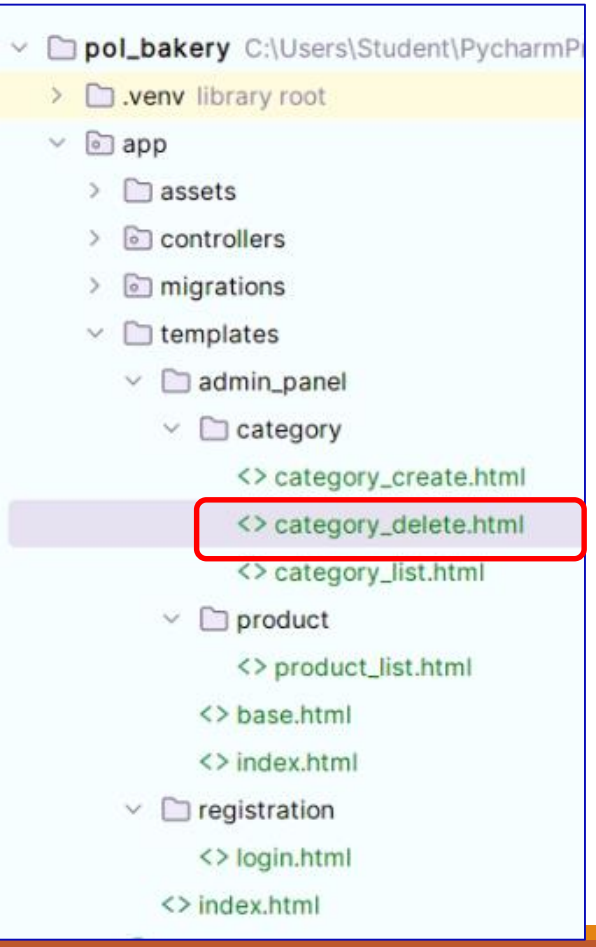
```
33

      1 usage
34    @login_required
35    def category_delete(request, pk):
36        if request.method == "POST":
37            if "delete_confirm" in request.POST:
                  try:
                      CategoryController.delete_category(pk)
                      return redirect(reverse("admin_panel:category_list"))
                  except Exception as e:
                      error = f"Ошибка удаления: {str(e)}"
              else:
                  error = "Подтвердите удаление"

              return render(request, template_name: "admin_panel/category/category_delete.html", context: {
                  "category_id": pk,
                  "error": error
              })

      return render(request, template_name: "admin_panel/category/category_delete.html", context: {
          "category_id": pk
      })
```

Удаление категории

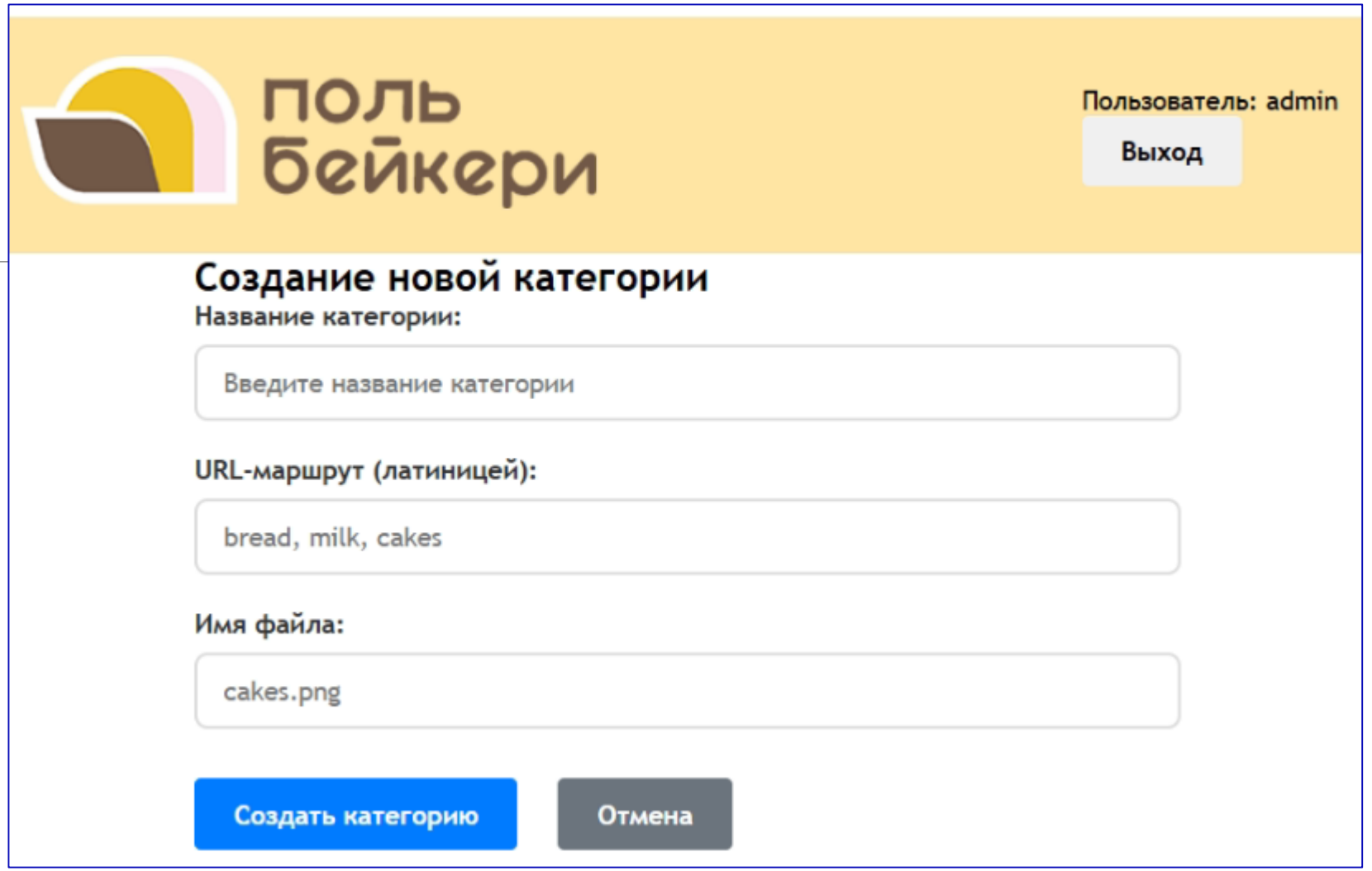


category_delete.html

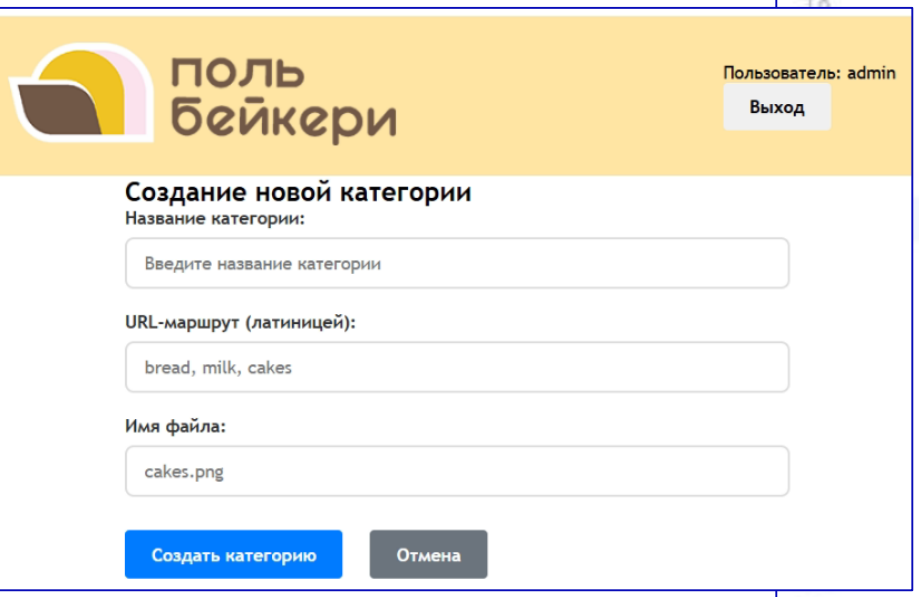
```
{%  extends 'admin_panel/base.html' %}
{% block title %}Удаление категории{% endblock %}
 {% block content %}
    <h2>Удалить категорию?</h2>
    <p><strong>ID категории:</strong> {{ category_id }}</p>
    {% if error %}
        <div class="error">
            <strong>Ошибка:</strong> {{ error }}
        </div>
    {% endif %}
    <form method="post">
        {% csrf_token %}
         <input type="hidden" name="delete_confirm" value="1">
        <button type="submit">Да, удалить навсегда</button>
        <a href="{% url 'admin_panel:category_list' %}">Отмена</a>
    </form>
{% endblock %}

```

Создание категории



Создание категории



<> category_create.html

```
7   <form method="post" class="form">
10      <div class="form-group">
11          <label for='{{form.name_category}}'>Название категории:</label>
12          {{ form.name_category }}
13          {% if form.name_category.errors %}
14          <div class="error">{{ form.name_category.errors }}</div>
15          {% endif %}
16      </div>
17
18      <div class="form-group">
19          <label for='{{form.route_category}}'>URL-маршрут (латиницей):</label>
20          {{ form.route_category }}
21          {% if form.route_category.errors %}
22          <div class="error">{{ form.route_category.errors }}</div>
23          {% endif %}
24      </div>
25      <div class="form-group">
26          <label for='{{form.category_image}}'>Имя файла:</label>
27          {{ form.category_image }}
28          {% if form.category_image.errors %}
29          <div class="error">{{ form.category_image.errors }}</div>
30          {% endif %}
31      </div>
32      <div class="buttons">
33          <button type="submit" class="btn btn-primary">Создать категорию</button>
34          <a href="{% url 'admin_panel:category_list' %}" class="btn btn-secondary">Отмена</a>
35      </div>
36  </form>
37
38  {% endblock %}
```

Создание категории

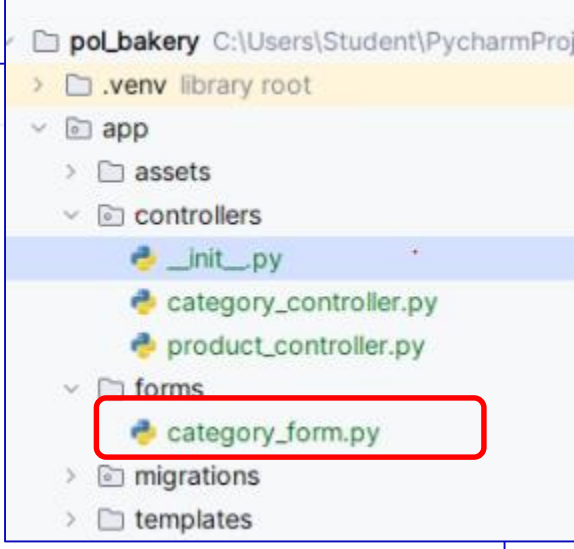
category_form.py

```python
from django import forms
3 usages  new *
class CategoryForm(forms.Form):
    name_category = forms.CharField(
        max_length=255,
        label="Название категории",
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'Введите название категории'
        })
    )
    route_category = forms.CharField(
        max_length=255,
        label="URL-маршрут (латиницей)",
        help_text="Только латинские буквы, цифры, дефисы",
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'bread, milk, cakes'
        })
    )
    category_image = forms.CharField(
        max_length=255,
        label="Введите имя файла",
        help_text="имя файла",
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'cakes.png'
        })
    )
```

views.py

```python
1 usage
@login_required
def category_create(request):
    if request.method == "POST":
        form = CategoryForm(request.POST)
        if form.is_valid():
            try:
                # Отправляем данные в API
                data = {
                    "name_category": form.cleaned_data["name_category"],
                    "route_category": form.cleaned_data["route_category"],
                    "category_image": form.cleaned_data["category_image"]
                }
                CategoryController.create_category(data)
                return redirect(reverse("admin_panel:category_list"))
            except Exception as e:
                form.add_error( field: None,  error: f"Ошибка API: {str(e)}")
    else:
        form = CategoryForm()

    return render(request,  template_name: "admin_panel/category/category_list.html",  context: {
        "form": form
    })
```



Продемонстрируйте результаты работы преподавателю!!